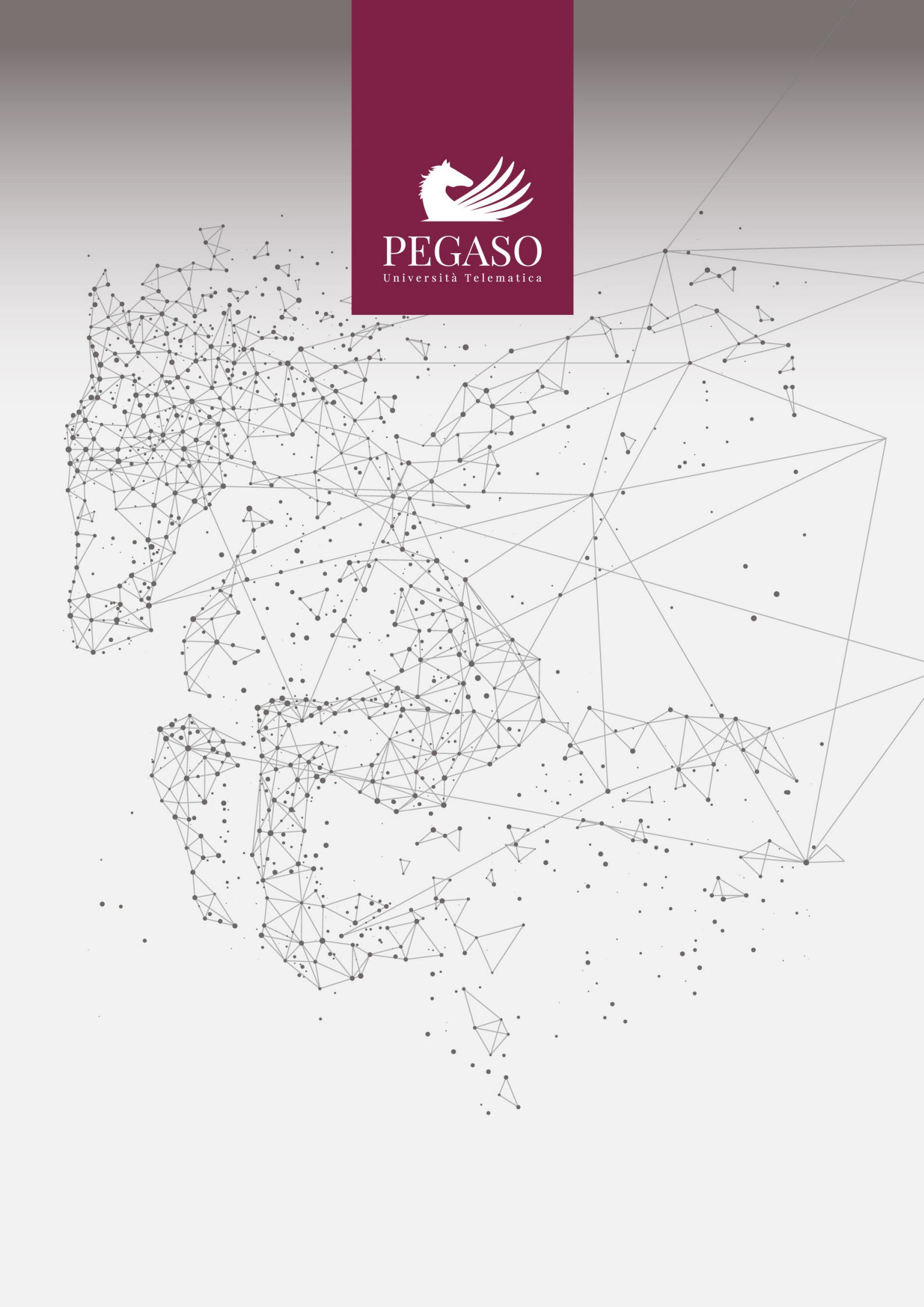

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. PYTHON E LE ESPRESSIONI REGOLARI....4
3. GRUPPI....10
4. ESERCIZI CON LE ESPRESSIONI REGOLARI....16
BIBLIOGRAFIA....19

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si approfondiscono gli strumenti che Python mette a disposizione per le espressioni regolari. In particolare, vengono illustrate le funzioni findall, finditer, sub e subn del modulo re. Vengono illustrati i pattern precompilati e le asserzioni. Si presentano, inoltre, i gruppi (con e senza cattura) e le loro funzionalità nelle ricerche a pattern con espressioni regolari. Infine, vengono illustrati quattro esercizi di esempio di uso delle espressioni regolari.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Python e le espressioni regolari

La funzione **findall**() del modulo re con il seguente formato:

```
findall(pattern, string, flags = 0)
```

trova **tutte le sottostringhe che corrispondono all'espressione regolare.** La funzione restituisce una **lista di sottostringhe** (in caso di nessuna corrispondenza la lista sarà vuota).

Nel codice sorgente che segue si riportano due esempi di uso della funzione findall(). Nel primo esempio, vengono ricercate tutte le parole senza distinzione maiuscole/minuscole, mentre nel secondo caso con la disizione maiuscole/minuscole.

```
import re

stringa = "123456 Questa stringa ha almeno una 1223 lettera 1233456"
ris = re.findall(r"[a-z]+", stringa , re.I)
print("Risultato della funzione findall:\n",ris)
print()
ris = re.findall(r"[a-z]+", stringa)
print("Risultato della funzione findall con distinzione \
maiuscole/minuscole:\n",ris)
```

Il risultato dell'elaborazione del programma precedente è il seguente:

```
Risultato della funzione findall:
 ['Questa', 'stringa', 'ha', 'almeno', 'una', 'lettera']

Risultato della funzione findall con distinzione maiuscole/minuscole:
 ['uesta', 'stringa', 'ha', 'almeno', 'una', 'lettera']
```

I quantificatori normali sono "golosi" (in inglese greedy), cioè cercano l'occorrenza più grande possibile. Ad esempio, il pattern ' ".*" ' permette di trovare tutte le parole racchiuse tra doppi apici. Tale pattern funziona, ma non in modo propriamente corretto come dimostrato nell'esempio seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import re

stringa = 'class="pluto" id="pippo"'
ris = re.findall(r'".*"', stringa, re.I)
print(ris)
```

Come è possibile osservare dal risultato dell'elaborazione, non vengono trovati due elementi ("pippo" e "pluto"), ma un unico elemento con i doppi apici all'inizio e alla fine.

```
['"pluto" id="pippo"']
```

È possibile rendere i quantificatori «non golosi» inserendo un punto interrogativo subito dopo il quantificatore come illustrato nel codice sorgente che segue:

```
import re

stringa = 'class="pluto" id="pippo"'
ris = re.findall(r'".*?"', stringa, re.I)
print(ris)
```

In questo caso, il programma funzionerà correttamente in modalità non greedy.

```
['"pluto"', '"pippo"']
```

La funzione **finditer**() del modulo re con il seguente formato:

```
finditer(pattern, stringa, flag)
```

a differenza della funzione findall() **restituisce anche l'indice** insieme alle stringhe corrispondenti. La funzione **start**() restituisce l'indice iniziale della corrispondenza trovata, la funzione **end**() restituisce l'indice finale della corrispondenza trovata e la funzione **group**() restituisce la corrispondenza trovata.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ad esempio, si vogliono conoscere gli indici delle stringhe corrispondenti alle cifre numeriche nella stringa:

```
import re
stringa = "Vado al mare 4 ore al giorno. Passeggio 2 ore al giorno."
pat = r'[0-9]'
for ris in re.finditer(pat, stringa):
    s = ris.start()
    e = ris.end()
    g = ris.group()
    print(g, "trovata in posizione [",s,",",e,"]")
```

Il risultato dell'elaborazione del programma precedente è il seguente:

```
4 trovata in posizione [ 13 , 14 ]
2 trovata in posizione [ 40 , 41 ]
```

La funzione **sub**() del modulo re con il seguente formato:

```
sub(pattern, sostituta, stringa, flags=0)
```

Sostituisce tutte le parti corrispondenti di una stringa con una stringa sostituta. È possibile limitare il numero di sostituzioni attraverso il parametro opzionale count. La funzione **subn**() ritorna anche il numero di sostituzioni.

Nell'esempio di uso della funzione sub() che segue, si vuole sostituire a tutte le parole e agli spazi, la seguente stringa "[…]".

```
import re

stringa = "123456 Questa stringa ha almeno una 1223 lettera 1233456"
ris = re.sub(r"[A-Za-z ]+", "[...]", stringa)
print("Risultato della funzione sub:\n",ris)
```

Il risultato dell'elaborazione del programma precedente è il seguente:

```
Risultato della funzione sub:
 123456[...]1223[...]1233456
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quando le espressioni regolari in Python vengono **compilate**, esse vengono convertite in **modelli.** Questi pattern sono in realtà **oggetti pattern** che contengono diverse funzioni per eseguire diversi compiti che possono includere la ricerca, la corrispondenza e la sostituzione. Una volta compilato un pattern, si può usare in un qualsiasi punto del programma. La funzione del modulo re per la compilazione di espressioni regolari è **compile**().

Si riporta un esempio di uso della funzione match() con **pattern precompilato** in cui si ricercano i numeri compresi in due stringhe diverse.

```
import re
pattern_comp = re.compile(r"\d+")

ris = pattern_comp.search("Lalalala 123")
print(ris)
print(ris.group())

ris2 = pattern_comp.search("234Lalalala 123456789")
print(ris2)
print(ris2.group())
```

Il risultato della chiamata a search() su modello precompilato con pattern "\d+" su due diverse stringhe è il seguente:

```
<re.Match object; span=(9, 12), match='123'>
123
<re.Match object; span=(0, 3), match='234'>
234
```

Per specificare delle espressioni regolari più complesse è possibile utilizzare le **asserzioni.**

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'**asserzione lookahed positiva** ha il seguente formato (?=pattern). Per esempio, Paolo (?=Rossi) corrisponderà con Paolo solo se seguito da Rossi come mostrato nel caso successivo.

```
['Paolo', 'Paolo', 'Paolo']
['Paolo ']
```

Il risultato dell'elaborazione del programma precedente mostrerà nel primo caso tutti e tre i Paolo, mentre nel secondo caso un solo Paolo corrispondente a Paolo Rossi.

L'**asserzione lookahed negativa** ha il seguente formato (?!pattern). Per esempio, Paolo (?!Rossi) corrisponderà con Paolo solo se **non** seguito da Rossi come mostrato nel caso successivo.

```
import re

stringa = 'Paolo Rossi Paolo Verdi Paolo Bianchi'
ris = re.findall(r'Paolo (?!Rossi)', stringa, re.I
print(ris)
```

Il risultato dell'elaborazione del programma precedente sarà due volte Paolo corrispondenti a Paolo Verdi e Paolo Bianchi:

```
['Paolo ', 'Paolo ']
```

L'**asserzione lookbehind positiva** ha il seguente formato (?<=pattern). Valida l'espressione successiva solo se la condizione pattern è verificata. Per esempio, trovare le parole che finiscono per 'sto' la cui lettere precedente sia 'a':

```
import re

testo = "cesto pasto fasto pesto costo"
print(re.findall(r'(\w+(?<=a)sto)', testo))
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione del programma precedente per la stringa di esempio riportata è:

```
['pasto', 'fasto']
```

L'**asserzione lookbehind negativa** ha il seguente formato (?<!pattern). Valida l'espressione successiva solo se la condizione pattern **non** è verificata. Per esempio, trovare le parole che finiscono per 'sto' la cui lettere precedente **non** sia 'a':

```
import re

testo = "cesto pasto fasto pesto costo"
print(re.findall(r'(\w+(?<!a)sto)', testo))
```

Il risultato dell'elaborazione del programma precedente per la stringa di esempio riportata è:

```
['cesto', 'pesto', 'costo']
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Gruppi

Cercare corrispondenze di modelli è la base delle potenti capacità fornite dalle espressioni regolari. L'aggiunta di **gruppi** ad un modello consente di isolare parte del testo corrispondente, espandendo queste capacità per creare un **parser (analizzatore morfologico e sintattico di un testo).**

Le espressioni regolari, infatti, sono spesso usate per suddividere stringhe scrivendo una espressione regolare divisa in **diversi sottogruppi che corrispondono a differenti componenti.** I gruppi sono definiti racchiudendo i modelli di corrispondenza tra **parentesi tonde.** Per esempio, è possibile ripetere il contenuto di un gruppo con un qualificatore di ripetizione *, +, ?, o {m,n}.

A titolo indicativo, (ab)* verificherà zero o più ripetizioni del gruppo "ab" come mostrato nel codice seguente:

```
import re

p = re.compile('(ab)*')
print(p.match('abababababab'))
```

Il risultato dell'elaborazione del programma precedente è il seguente:

```
<re.Match object; span=(0, 10), match='abababababab'>
```

I gruppi indicati con ( ) catturano anche l'inizio e la fine dell'indice del testo che verificano. I gruppi vengono numerati partendo da 0 come dimostrato dal codice seguente:

```
import re

p = re.compile('(a)b')
m = p.match('ab')
print(m.group())
print(m.group(0))
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione del programma precedente è il seguente:

```
ab
ab
```

I sottogruppi sono numerati da sinistra a destra, da 1 in poi. I gruppi possono, inoltre, essere annidati. Per determinarne il numero, è necessario contare il numero di parentesi aperte, procedendo da sinistra a destra. Nel codice seguente si utilizzano due sottogruppi:

```
import re

p = re.compile('(a(b)c)d')
m = p.match('abcd')
print(m.group(0))
print(m.group(1))
print(m.group(2))
```

Il risultato dell'elaborazione del programma precedente prevede il frammento totale uguale a "abcd", mentre il primo gruppo uguale a "abc" e il secondo gruppo uguale a "b":

```
abcd
abc
b
```

Al metodo group() possono essere passati più valori di gruppo per volta. In questo caso, la funzione restituirà una tupla contenente il corrispondente valore dei gruppi indicati:

```
import re

p = re.compile('(a(b)c)d')
m = p.match('abcd')
print(m.group(2,1,2))
```

La tupla risultante dalla chiamata a group() passando più valori di gruppo sarà:

```
('b', 'abc', 'b')
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il metodo groups() restituisce una tupla contenente la stringa di tutti i sottogruppi, **da 1 in poi.** Il gruppo **0** rappresenta la stringa corrispondente all'intera espressione.

```
import re

p = re.compile('(a(b)c)d')
m = p.match('abcd')
print(m.groups())
```

La tupla restituita dal metodo groups() sarà uguale a:

```
('abc', 'b')
```

I parametri in un modello permettono di specificare che il contenuto di un precedente gruppo catturante **deve essere trovato anche nella posizione corrente all'interno della stringa.** Per esempio, \1 avrà successo se l'esatto contenuto del gruppo 1 si troverà nella posizione corrente, altrimenti fallirà. Questa funzionalità è molto utile per effettuare delle sostituzioni nelle stringhe.

Per esempio, la seguente espressione regolare intercetta parole doppie all'interno di una stringa:

```
import re

p = re.compile(r'(\b\w+)\s+\1')
print(p.search('RE trovata nella nella stringa').group())
```

La parola doppia intercettata sarà, infatti:

```
nella nella
```

È possibile anche fornire dei nomi ai gruppi secondo il formato (?P<nome_gruppo>…). Il metodo group() che si occupa di tutti i gruppi catturanti accetta o interi, per riferirsi ai gruppi tramite numero, o stringhe contenenti il nome del gruppo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Gruppi con nome restituiscono ancora numeri, in modo che sia possibile ottenere informazioni circa un gruppo in due modi. Nell'esempio che segue il gruppo "parola" corrisponde al gruppo numero 1.

```
import re

p = re.compile(r'(?P<parola>\b\w+\b)')
m = p.search( '(((( Molta punteggiatura )))' )
print(m.group('parola'))
print(m.group(1))
```

Il risultato delle due chiamate alla funzione print() sarà:

```
Molta
Molta
```

Il caso mostrato in precedenza di individuazione della ripetizione di una parola nella stringa, grazie alla denominazione dei gruppi diventa:

```
import re

p = re.compile(r'(?P<parola>\b\w+)\s+(?P=parola)')
ris = p.search('Re trovata nella nella stringa')
print(ris)
print(ris.group())
```

Il risultato dell'elaborazione del programma precedente è, quindi:

```
<re.Match object; span=(11, 22), match='nella nella'>
nella nella
```

Si riporta, ora, un esempio di utilizzo di pattern precompilati con gruppi opportunamente nominati ("prima_parola", "ultima_parola", "parola", "altra_parola", "finisce _con_o").

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import re

stringa = 'Questa è una porzione di testo -- con punteggiatura.'

for pattern in [ r'^(?P<prima_parola>\w+)',
                 r'(?P<ultima_parola>\w+)\S*$',
                 r'(?P<parola_t>\bt\w+)\W+(?P<altra_parola>\w+)',
                 r'(?P<finisce_con_o>\w+o)\b',
                 ]:
    pattern_compilato = re.compile(pattern)
    match = pattern_compilato.search(stringa)
    print("Corrispondenza",pattern)
    print("Groups:",match.groups())
    print()
```

Il risultato dell'elaborazione del programma precedente è il seguente:

```
Corrispondenza ^(?P<prima_parola>\w+)
Groups: ('Questa',)

Corrispondenza (?P<ultima_parola>\w+)\S*$
Groups: ('punteggiatura',)

Corrispondenza (?P<parola_t>\bt\w+)\W+(?P<altra_parola>\w+)
Groups: ('testo', 'con')

Corrispondenza (?P<finisce_con_o>\w+o)\b
Groups: ('testo',)
```

Un gruppo senza cattura è identico ad un gruppo con cattura, ma a differenza di questo non viene recuperato il suo contenuto. Il gruppo non-catturante viene identificato dalla seguente sintassi (?:...). Tra le parentesi dovranno essere inserite le espressioni regolari. Nel codice seguente si individua lo stesso pattern per il gruppo, con la sola differenza che nel primo caso il gruppo è catturante, mentre nel secondo caso è senza cattura.

```
import re

m = re.match("([abc])+", "abc")
print(m)
print(m.groups())
m = re.match("(?:[abc])+", "abc")
print(m)
print(m.groups())
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Come è possibile osservare dal risultato visualizzato a schermo, il metodo groups nel primo caso restituisce una 'c', mentre nel secondo caso non restituisce nulla.

```
<re.Match object; span=(0, 3), match='abc'>
('c',)
<re.Match object; span=(0, 3), match='abc'>
()
```

Le parentesi che racchiudono un gruppo senza cattura hanno la stessa funzione delle parentesi delle espressioni aritmetiche. Per esempio, r"cab+" individua una sottostringa che inizia con "ca", seguita da almeno da una "b". Il pattern r"c(?:ab)+" individua una sottostringa che inizia con una "c", seguita da una o più "ab". Si sottolinea che non vengono inseriti spazi tra "(?: " e il resto dell'espressione regolare.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Esercizi con le espressioni regolari

Si propongono, ora, quattro esercizi con le espressioni regolari.

Esercizio n. 1 – cercare nella stringa "cane crotalo canarino criceto cervo daino dromedario" tutti gli animali che cominciano per c seguita da una vocale.

```
import re

testo = "cane crotalo canarino criceto cervo daino dromedario"
print(re.findall(r'\b(c(?=[aeiou])\w+)', testo))
```

Il risultato del programma precedente sarà correttamente il seguente (in crotalo e criceto la seconda lettera non è una vocale):

```
['cane', 'canarino', 'cervo']
```

Esercizio n. 2 – cercare nella stringa "cane crotalo canarino criceto cervo daino dromedario" solo le parole che iniziano con la c solo se non seguite da una vocale.

```
import re

testo = "cane crotalo canarino criceto cervo daino dromedario"
print(re.findall(r'\b(c(?![aeiou])\w+)', testo))
```

Il risultato del programma precedente sarà correttamente il seguente:

```
['crotalo', 'criceto']
```

Esercizio n. 3 – sostituire nelle righe in cui è presente una data dal formato europeo (gg-mm-aa) a quello americano (mm-gg-aaaa). Per semplicità si considerano tutte le foto scattate dopo il 2000.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Esempio: 31-01-10_foto32

Pattern: \d{2}-\d{2}-\d{2}_foto.* trova le righe

Sostituzione delle righe trovate con

{Gruppo2}–{Gruppo1}-20{Gruppo3}_backup{Gruppo4}

che in Python diventa

\2-\1-20\3_foto\4

```
import re
testo = " 31-01-10_foto32 \n 24-01-10_foto1 \n 24-02-10_foto_mona \n \
11-03-09_foto_lisa"

print("Testo con date in formato: gg-mm-aa:\n" + testo)

testo = re.sub(r'(?P<gg>\d{2})-(?P<mm>\d{2})-(?P<aa>\d{2})_foto(?P<final>.*)',\
               r'\2-\1-20\3_foto\4', testo)

print("Testo con date in formato: mm-gg-aaaa: \n" + testo)
```

Il risultato dell'elaborazione del programma precedente è:

```
Testo con date in formato: gg-mm-aa:
 31-01-10_foto32
 24-01-10_foto1
 24-02-10_foto_mona
 11-03-09_foto_lisa
Testo con date in formato: mm-gg-aaaa:
 01-31-2010_foto32
 01-24-2010_foto1
 02-24-2010_foto_mona
 03-11-2009_foto_lisa
```

Esercizio n. 4 – verificare un indirizzo e-mail nella forma ad esempio nome_utente@dominio.it seguendo i seguenti vincoli:

- il nome_utente può contenere lettere, numeri, underscore, trattini, ma deve iniziare con una lettera - porzione del pattern corrispondente **[a-zA-Z][\w-]***;
- al nome_utente deve seguire una chiocciola **[a-zA-Z][\w-]*@** ;
- il dominio può contenere solo lettere **[a-zA-Z]+** seguite da un punto e da altre lettere (eventualmente ci possono essere altri sottodomini) – **(\.[a-z]{2,3})+**

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import re

email = 'dz@dominio.aa.it az@dominio.com bz@.it @dominio.it cz@ cz@dominio'
token = email.split(' ')
pattern = r'[a-zA-Z][\w-]*@[a-zA-Z]+(\.[a-z]{2,3})+'
for stringa in token:
    print(stringa,end=' - ')
    ris = re.search(pattern, stringa)
    if ris:
        print("è una mail valida")
    else:
        print("non è una mail valida")
```

Il risultato dell'elaborazione del programma precedente verifica tutte le mail contenute nella stringa:

```
dz@dominio.aa.it - è una mail valida
az@dominio.com - è una mail valida
bz@.it - non è una mail valida
@dominio.it - non è una mail valida
cz@ - non è una mail valida
cz@dominio - non è una mail valida
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev, D. (2019). Data Science con Python. Apogeo.
- Kuchling, A.M. (2021). HOWTO sulle Espressioni Regolari

  https://www.python.it/doc/howto/Regex/regex-it/regex-it.html

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).